Fine-Grained Complexity 20224191, Problem Set 1 **2026**
Released: Thursday, November 12, 2026
Due date: Sunday, November 29, 2026, 22:00 (Israel time)
Please send your solutions to Ron Safier at `ron.safier@weizmann.ac.il`.

Notation and preliminaries: We use the notation $[n] = \{1, \ldots, n\}$. Unless explicitly stated, an algorithm is allowed to be randomized (and for all inputs x succeeds with probability $\geq 2/3$).

1 Amplifying the Success Probability of Randomized Algorithms

In this question we will see how to turn an algorithm that makes two-sided error with probability $1/3$ *into one that (a) has much smaller error probability and (b) only has one-sided error.*

Let $k \in \{3, 4, 5, \ldots\}$ be a fixed constant. A k-clique in a graph is a set of k (distinct) nodes such that all pairs are neighbors. Suppose that there is a randomized algorithm $\mathcal{A}$ that decides if a graph on n nodes contains a k-clique in $O(T(n))$ time and is correct with probability at least $2/3$.

(a) Design an algorithm $\mathcal{A}'$ that takes $O(T(n) \cdot \log n)$ time and errs only with probability at most $1/n^{10}$.
(*Hint:* Repeat and take the majority. Use a Chernoff bound for the analysis.)

(b) Design an algorithm $\mathcal{A}''$ that takes $O(T(n) \cdot \log^2 n)$ time and errs only with probability $1/n^9$, and in addition, must be correct when it outputs *yes*.
(*Hint:* Use an idea we saw in class to obtain a witness. Use a Union Bound for the analysis.)

Remark You are encouraged to read (and use anything from) this short *primer to randomness* from the fine-grained complexity course at Max-Planck Institute: (click here).

2 Convolution 3SUM to 3SUM

Prove that if 3-SUM on n numbers in $[-U, +U]$ can be solved in $T(n, U)$ time, then the following problem can be solved in $T(O(n), O(nU))$ time.

Convolution 3SUM: Given three arrays A, B, C each containing n numbers in $[-U, +U]$ decide if there are two numbers $i, j \in [n]$ such that $A[i] + B[j] + C[i + j] = 0$.

3 Triangle Detection to 3SUM

A triangle in a graph is a 3-clique. Prove that if 3SUM can be solved in $T(n, U)$ time then one can detect if a graph on n nodes and m edges has a triangle in $O(\log n) \cdot T(O(m), O(n^3))$ time.

4 3SUM to Sumset Size

The *sumset* $X + Y$ of two sets of integers X, Y is the set of all pairwise sums $X + Y = \{x + y \mid x \in X, y \in Y\}$. The size of the sumset $|X + Y|$ is at most n^2 but could be smaller.

Prove that Sumset Size is 3SUM-Hard: if we can compute the size of the sumset of two sets in $O(n^{2-\varepsilon})$ time, for some $\varepsilon > 0$, then the 3SUM Conjecture is false.

Bonus: Open Questions

The following are open research questions, included as optional bonus problems.

Bonus 1: C_4 Detection

Problem definition. Given an undirected graph G on n vertices, decide whether it contains a cycle on four distinct vertices as a subgraph. The cycle need not be induced: edges between opposite vertices are allowed.

Background. There is a simple $O(n^2)$-time algorithm. For each vertex v, enumerate paths $v - u - w$ with $w \neq v$ until two different middle vertices u, u' reach the same endpoint w. This gives the cycle v, u, w, u'. Before such a collision, each endpoint is encountered at most once, so the exploration takes $O(n)$ time per vertex.

Open question. Finding a fine-grained reduction either *from C_4 Detection to 3SUM* or *from 3SUM to C_4 Detection* would be very interesting. A direct subquadratic-time algorithm for C_4 Detection would also be a great result. In the standard word-RAM model with $O(\log n)$-bit words, even an $o(n^2)$-time algorithm that only shaves low-order terms from the quadratic bound is not known.

A particularly interesting special case is a d-regular graph with $d = \Theta(\sqrt{n})$, for example $d \approx \sqrt{n}/2$. Can one obtain a subquadratic-time algorithm for this case?

Bonus 2: Three Points on a Line

Problem definition. Given n distinct points in the plane, decide whether three of them lie on a common line.

Background. A simple $O(n^2 \log n)$-time algorithm considers each point p and sorts the other points by the slope of the line they determine with p. Two equal slopes identify three collinear points. With additional geometric techniques, the running time can be improved to $O(n^2)$.

There is a familiar reduction *from 3SUM to Three Points on a Line*: map each number x to (x, x^3). For distinct x, y, z, the identity

$$\frac{y^3 - x^3}{y - x} - \frac{z^3 - x^3}{z - x} = (y - z)(x + y + z)$$

shows that the three corresponding points are collinear if and only if $x + y + z = 0$.

Open question. Either solving Three Points on a Line directly in $O(n^{2-\varepsilon})$ time for some fixed $\varepsilon > 0$, or finding a fine-grained reduction *from Three Points on a Line to 3SUM*, would be a great result.